**Faculty of Engineering – Ain Shams University**
**Mechatronics Engineering Department**

**MCT411 (UG2018): Hybrid Control Systems**
**MCT413s (UG2023): Control Systems Design (2)**
**MCT414 (UG2023): Modern Control**

***Fall 2026***

*Major Task Project*

# Furuta Pendulum

**(Rotary Inverted Pendulum) — Modelling, Hybrid & Control, and Real-Time Dashboard**

## 1. Introduction

The Furuta pendulum (rotary inverted pendulum) is one of the most widely used benchmark platforms in control engineering education and research. It consists of a horizontal driven arm rotating about a fixed vertical axis (angle $\theta_0$) with a free pendulum link attached at its end (angle $\theta_1$), as shown in Figure 1. The system is underactuated (one actuator, two degrees of freedom), highly nonlinear, and open-loop unstable around its upright equilibrium, which makes it an excellent case study for theoretical modelling, stability analysis, and the design and comparison of classical, state-space and optimal controllers.

This project is jointly assigned to MCT411 (Hybrid Control Systems), MCT414 (Modern Control), and MCT413s (Control Systems Design (2)). The project covers the modelling, pole-placement and LQR (state-feedback / optimal control) design methodology used to stabilize the pendulum about its upright position, and also focuses on the hybrid nature of the overall task — the discrete-event switching logic (implemented in Stateflow) that governs transitions between the swing-up stage (a discrete/continuous hybrid behaviour) and the continuous-time stabilization stage. Students are expected to integrate both perspectives into a single, coherent hybrid control system.

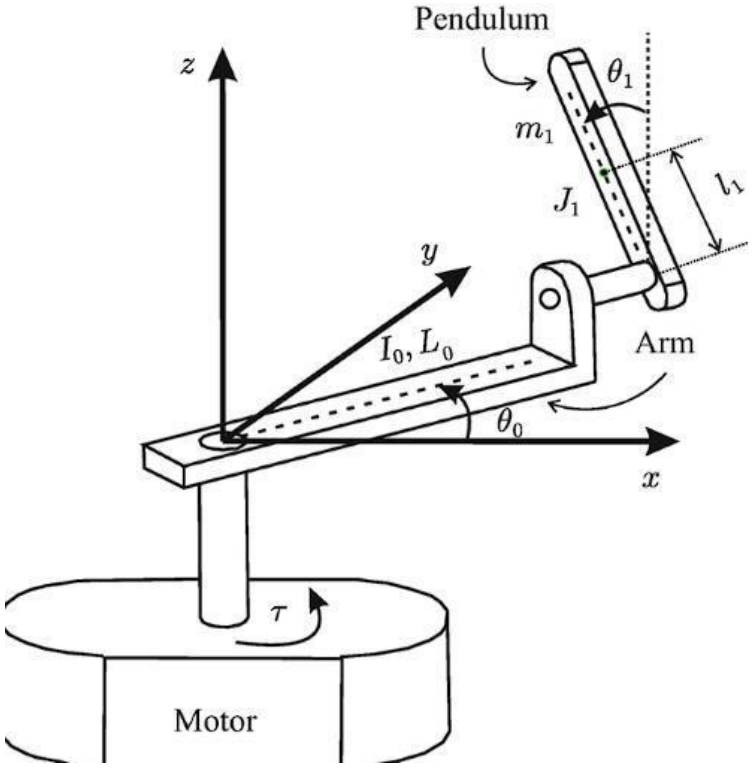


*Figure 1 — Furuta (rotary inverted) pendulum: arm angle $\vartheta 0$, pendulum angle $\vartheta 1$, motor torque $\tau$.*

## 2. Applications of the Furuta Pendulum / Underactuated Balance Control in Real Life

Although the Furuta pendulum itself is a laboratory benchmark, the modelling and control techniques it teaches — stabilizing an unstable, underactuated system about an equilibrium using state feedback, optimal control and hybrid switching — are directly transferable to numerous real engineering systems, including:

- Self-balancing personal transporters and two-wheeled robots (e.g., Segway-type vehicles), which balance a body about an unstable upright equilibrium using the same inverted-pendulum principle.
- Humanoid and bipedal robot balance and gait control, where posture stabilization relies on inverted-pendulum models (e.g., the Linear Inverted Pendulum Model used in walking robots).
- Rocket and launch-vehicle attitude / thrust-vector control, and vertical-landing boosters, which must actively stabilize an inherently unstable body about the vertical using real-time feedback control.
- Crane load-sway suppression (tower cranes, gantry and overhead cranes, ship-mounted cranes), where the suspended load behaves like a pendulum that must be actively damped during positioning.
- Segbots, balancing wheelchairs and mobile-robot platforms used in warehouses and last-mile delivery.
- Ship and offshore-platform stabilization, and satellite/spacecraft attitude control, which share the underactuated stabilization structure and benefit from LQR/optimal control design.
- Postural-control studies in biomechanics, rehabilitation robotics and prosthetics, where the human body's standing balance is frequently modelled as an inverted pendulum.
- Industrial and commercial control-education platforms (e.g., Quanser QUBE-Servo, Educational Control Products ECP Rotary Inverted Pendulum) used worldwide to teach and benchmark new control algorithms before deployment on real hardware.

Understanding this platform therefore equips students with transferable skills for a wide range of robotics, aerospace, automotive and industrial-automation control problems.

## 3. Team Formation

- Each team must consist of 6 to 8 students.
- All members of a team must be registered in the same section.
- Each team must submit a team-formation sheet (names, IDs, section, and a nominated team leader) to the course instructor by the date announced in the course schedule.
- Given the larger team size, suggested role distribution (roles may overlap and should be documented in the technical report's contribution section):
  - Team lead & system integration
  - Theoretical modelling & stability analysis (State space, Euler-Lagrange derivation, linearization)
  - Classical / state-feedback controller design (PID and pole placement) & tuning
  - Optimal controller design (LQR) & tuning, and hybrid swing-up / Stateflow logic
  - Mechanical design, hardware assembly and wiring (motor, encoders, electronics)
  - Dashboard, GUI and digital-twin development (MATLAB App Designer / Simulink Dashboard / animation)

- Testing, performance-metrics evaluation, documentation and video production

## 4. Project Scope and Technical Requirements

### 4.1 Theoretical Analysis and Modelling

- Derive the nonlinear equations of motion of the rotary inverted pendulum (e.g., using the Euler-Lagrange method), clearly stating all assumptions (rigid links, friction model, motor dynamics).
- Identify / estimate the physical parameters of your own hardware (arm length L0, pendulum length l1, masses, inertias J0/J1, motor and gearbox constants, friction coefficients) and document the identification method used.
- Linearize the nonlinear model about the upright (unstable) equilibrium and obtain the state-space representation.
- Build a simulation model preferably using MATLAB Simscape / SimMechanics (multibody model) in addition to the analytical state-space model, and validate that the two are consistent.

### 4.2 Stability Analysis

- Analyze the open-loop equilibrium points (stable/unstable, downward/upright) and their local stability using the linearized model (eigenvalues of the state matrix A).
- Check controllability and observability of the linearized system before proceeding to controller design.
- Discuss closed-loop stability of each designed controller (e.g., closed-loop pole locations, gain and phase margins for the PID case).

### 4.3 Controller Design and Tuning — minimum 3 controllers required for stabilization and 1 for swing up

- PID controller — design and tune (e.g., using linear analysis, Ziegler-Nichols as a starting point, then fine-tuning) to stabilize θ1 at zero (upright).
- Pole-placement (full-state feedback) controller — select closed-loop pole locations to meet the required transient specifications and compute the feedback gain matrix K.
- LQR (Linear Quadratic Regulator) controller — select Q and R weighting matrices, justify the selection, and compute the optimal gain K.
- Swingup: implement an energy-based (or equivalent) swing-up controller that brings the pendulum from the hanging position into the capture region of the stabilizing controller.
- For every controller: present the design method, the final gains/parameters, and simulation results before implementing it on hardware.

### 4.4 Hybrid Control Architecture (Stateflow)

Implement the overall controller as a hybrid system using MATLAB Stateflow, combining:

- A discrete-event supervisory layer with at least two states — Swing-Up and Stabilization (a Balanced/Upright state) — plus transition guards based on θ1 and its rate (e.g., enter Stabilization once |θ1| and its angular velocity fall inside a capture region).
- Continuous-time / discrete-time control laws executed within each state (the swing-up law in one state, and the selected stabilizing controller — PID, pole placement or LQR — in the other).

- Safety transitions (e.g., automatic fall-back to Swing-Up or to a safe-stop state if θ1 exceeds a safe bound), demonstrating the discrete-event / continuous-time hybrid modelling concepts taught in MCT411.

### 4.5 Digital Twin

- Build an animated 3D (or 2D schematic) digital twin of the Furuta pendulum — using Simscape Multibody, Simulink 3D Animation, or an equivalent animated model — that mirrors the arm and pendulum angles in real time.
- The digital twin must be driven simultaneously by the same sensor signals and/or control action as the physical prototype during Hardware-in-the-Loop (HIL) operation, so the animated model visibly reproduces the real system's motion side by side with the hardware.

### 4.6 Real-Time Dashboard

- Develop a MATLAB dashboard/GUI (App Designer, Simulink Dashboard blocks, or equivalent) that displays in real time: the pendulum angle θ1, the arm angle θ0, the actuator (motor) control signal u, and any additional sensor/state signals relevant to your design (e.g., angular velocities, current, controller mode/state).
- Embed the animated digital twin (Section 4.5) within the same dashboard so both the raw signals and the visual/animated response are visible together.
- The dashboard should also indicate which hybrid state (Swing-Up / Stabilization) is currently active.

### 4.7 Performance Assessment and Benchmarking

Evaluate and compare the three (or four, with swing-up) controllers using quantitative performance metrics suitable for an inverted-pendulum stabilization task, including at minimum:

- Transient response: settling time, rise time, percentage overshoot, and peak time of θ1 after a disturbance or after swing-up capture.
- Steady-state error of θ1 and θ0 (as applicable).
- Error-integral indices: IAE, ISE and/or ITAE for a standardized disturbance test.
- Disturbance rejection: apply a repeatable manual/impulse disturbance to the pendulum and compare recovery time and maximum deviation across controllers.
- Control effort: peak and RMS actuator signal (u) required by each controller, and any actuator saturation observed.
- Robustness: repeat key tests at least twice and report consistency; discuss sensitivity to parameter/model uncertainty.
- Present all of the above as a comparison table and supporting response plots (simulation vs. experimental) for PID vs. pole placement vs. LQR.

## 5. Hardware and Implementation Guidelines

- Use one motor equipped with an encoder to control/measure θ0 (arm), and a separate encoder with a resolution of at least 360 pulses/revolution to measure θ1 (pendulum).
- All wiring should be routed and hidden as neatly as possible, and every mechanical/electrical component must be securely and rigidly fixed.

- You may use any embedded platform that can be configured as a MATLAB/Simulink hardware target (e.g., Arduino family boards, NXP FRDM-KL25Z, TI Delfino F28379D/F2837xD, STM32-based boards, or an equivalent supported target).
- The model must be built and controlled from MATLAB/Simulink (Stateflow charts for the hybrid supervisory logic as described in Section 4.4).
- You must implement genuine Hardware-in-the-Loop (HIL) operation: the controller (including the Stateflow state machine) runs against the real hardware in real time, not only in offline simulation.
- Include appropriate safety measures: an accessible emergency-stop/disable action, mechanical end-stops or soft limits, and clearance around moving parts during testing/demonstration.

## 6. Standard Control Design Procedure

Follow the standard, systematic procedure below when designing each controller, and reference it explicitly in your technical report:

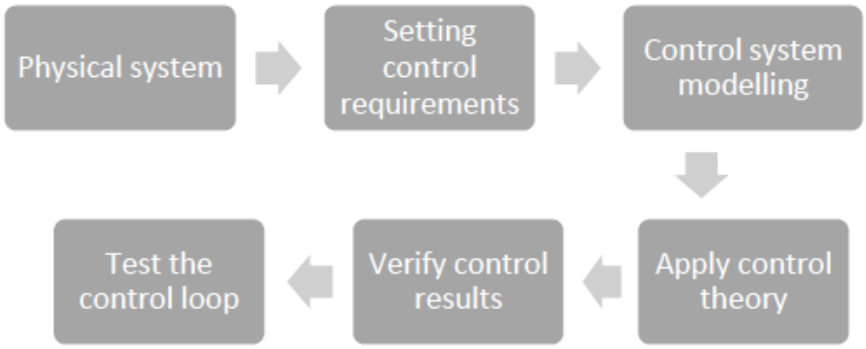



*Figure 2 — Standard control-system design procedure to follow for each controller.*

## 7. General Guidelines

- Clearly state all modelling assumptions and document the parameter-identification procedure used for your specific hardware.
- Keep consistent notation across the report, block diagrams and Simulink model (match Figure 1: θ0, θ1, L0, l1, J0, J1, τ).
- Organize Simulink/Stateflow models into clearly named, commented subsystems (plant, controller(s), supervisory Stateflow chart, dashboard/digital twin, data logging).
- Keep dated backups/version history of your model and code, and back up hardware configuration settings.
- Log each member's individual contribution throughout the project; this log is a mandatory section of the technical report.
- Cite every external reference, tutorial, dataset or code snippet used (including any of the video resources listed in Section 9).
- Test disturbance-rejection and robustness scenarios safely, with a clear emergency-stop procedure in place before any live demonstration.

## 8. Deliverables, Submission Phases and Grading

### 8.1 Submission Phases

- Phase 1 (Week 6): Deliver a MATLAB model of the pendulum system (preferably using Simscape/SimMechanics) together with open-loop dynamic validation of your Furuta pendulum design.
- Phase 2 (Week 9): Deliver the full stability analysis and the three controller designs (PID, pole placement, LQR) with simulation results and closed-loop pole/eigenvalue justification.
- Phase 3 — Final Submission (Week 13): Full project delivery, including the working hardware prototype with HIL implementation, the hybrid Stateflow logic, the real-time dashboard with integrated digital twin, and the complete technical report, per the grading table below.

### 8.2 Final Submission Form and Grading Scheme

| # | Deliverable | Weight |
|---|---|---|
| 1 | Working hardware prototype with a GUI/dashboard displaying pendulum angle (θ1), arm angle (θ0) and actuator signal (u).<br>The prototype shall demonstrate robustness against manual disturbances. | **20%** |
| 2 | Digital twin integrated into the dashboard: animated model synchronized in real time with the physical system and the same control action. | **10%** |
| 3 | **Technical report including:**<br>• Contribution of each team member<br>• Modelling and stability analysis of the pendulum system<br>• Controller design methods, requirements and tuning (PID / pole placement / LQR)<br>• Hybrid (Stateflow) implementation details<br>• Simulation results and performance-metrics comparison across controllers<br>• Detailed analysis of simulation vs. actual (experimental) results | **20%** |
| 4 | Oral discussion / defense of the design, simulation and results. | **15%** |
| 5 | Simulation results and controller implementation (Simulink model, all three controllers plus the Stateflow hybrid supervisory logic). | **20%** |
| 6 | Demo video (5 minutes max) that contains:<br>• Demonstration of the working prototype (incl. dashboard and digital twin)<br>• Brief discussion of controller design and simulation | **15%** |
| | Successful energy-based swing-up controller integrated with the hybrid state machine, demonstrated on real hardware. | |

*Note: using a PID-only design without pole placement and LQR, or omitting the hybrid Stateflow swing-up/stabilization switching logic, will result in deducted points, as this project explicitly targets hybrid and optimal control design.*

## 9. Useful Resources and Reference Videos

The following videos are provided as a guide to help you build, model, simulate and control your own rotary inverted pendulum. They illustrate possible mechanical/electrical designs and control approaches; you are expected to develop and justify your own design rather than copy one directly, and must cite any resource you draw on.

### 9.1 Build Examples and Tutorials

- **Ben Katz — "Furuta Pendulum" —** https://www.youtube.com/watch?v=XKzzWe15DEw
- **Robotics GEE — "Rotary Inverted Pendulum Control using LQR | STM32 Embedded Control System | Hardware Demo" —** https://www.youtube.com/watch?v=1rarHSgbDhA
- **Faisal Maniar — "Rotary Inverted Pendulum" —** https://www.youtube.com/watch?v=rhg0wcAvvhc
- **Henrique Ferrolho — "I built this £4,500 pendulum for £20" —** https://www.youtube.com/watch?v=rKChjuuR7K8
- **David Gonzalez — "Building another Furuta Pendulum" —** https://www.youtube.com/watch?v=1GOXuu1OZ3k
- **Cillk 11 — "Rotary Inverted Pendulum (PID) — Design, Build, Model, Swing Up and Stabilisation" —** https://www.youtube.com/watch?v=bY4t6yfBA24
- **Whisky Nguyễn — "Rotary Inverted Pendulum" —** https://www.youtube.com/watch?v=S5DLs4sHmQI

### 9.2 Simulation / Modelling

- **Robotics GEE — "Rotary Inverted Pendulum Control using LQR | STM32 Embedded Control System | Hardware Demo" (modelling & simulation walkthrough) —** https://www.youtube.com/watch?v=1rarHSgbDhA

### 9.3 Sim-to-Real and Digital Twins

- **Kevin Wood | Robotics & AI — "Reinforcement Learning on Hardware from Sim-to-Real (Rotary Inverted Pendulum)" —** https://www.youtube.com/watch?v=PTsIg7wUC9U

### 9.4 Examples of Previous Ain Shams University Mechatronics Student Projects

- **"Balancing the Furuta Pendulum | Rotary Inverted Pendulum" —** https://www.youtube.com/watch?v=D3wFRbZpMtU
- **"Rotary Inverted Pendulum — MCT411 — Hybrid Control Project Final Submission (Team 14)" —** https://www.youtube.com/watch?v=1aN-A-AvBck
- **"Furuta Pendulum — MCT411 Hybrid Control Systems" —** https://www.youtube.com/watch?v=ilNLgx1Jl00

## 10. Academic Integrity

Reference videos and prior student projects are provided strictly as guidance on mechanical design, wiring practice and controller implementation. Each team must perform its own parameter identification, modelling, controller design and tuning for its own hardware, and must clearly cite any external material used. Copying another team's (or a previous year's) analysis, code, or report without attribution constitutes academic misconduct and will be handled according to the Faculty's academic integrity policy.

*— End of Project Description —*